

dasein · 2026-09

CS 成长路线图

后端开发 · 基础架构 / AI Infra · 高性能计算 / 机器学习 · Agent

大二起点 | 四个学期的时间表 | 每个方向的验收标准

目录

总览：怎么用这份路线图	3
0. 一句话策略	3
1. 硬性时间锚点（倒推规划）	3
2. 文档地图	3
3. 每周节奏模板（时间充裕版，约 25-30h）	4
4. 关于「三个方向不知道选哪个」	4
方向选择：三方向轮转公测	5
0. 为什么不要现在选	5
1. 轮转公测的三个探针	5
2. 评分表（做完 3 个探针后填，凭当时的真实感受）	5
3. 三种合理结局（都不是失败）	6
4. 这 6 周的额外产出（跨方向通用，写进简历）	6
5. 常见反模式	6
共同底座：CSAPP 与系统基础	7
1. 学习顺序（有依赖关系的正确拓扑序）	7
2. C 语言：为什么绕不开，怎么学	7
3. CSAPP 正确读法（重点章节 + lab 优先级）	7
4. 操作系统：OSTEP vs 6.S081（二选一）	8
5. 网络： CS144 （强烈建议全做）	8
6. 数据库 / 存储： CMU 15-445	8
7. 分布式： MIT 6.824 / 6.5840	8
8. 数学与体系结构（方向 B/C 专用，方向 A 可后置）	8
9. 底座的时间预算（别让它吞掉主线）	9
主线 A：后端开发 · 基础架构	10
1. 能力分层（自下而上，缺一层就会在面试里露馅）	10
2. 阶梯与产出	10
3. 必须能答出来的面试题（自测清单）	10
4. 语言选择	11
5. 资源	11
6. 方向 A 的毕业作品（大二下-大三上完成）	11
主线 B：AI Infra · 高性能计算	12
1. 能力地图	12
2. 阶梯与产出	12
3. 硬件现实（决定你能走多远）	13
4. 自测清单（面试高频）	13
5. 资源	13
6. 方向 B 的毕业作品	13
主线 C：机器学习 · Agent	14
1. 能力地图	14
2. 阶梯与产出	14
3. 自测清单	15
4. 资源	15
5. 方向 C 的毕业作品	15
6. 方向 C 的最大陷阱	15
周计划与里程碑（大二上 → 秋招）	16
学期总览	16
大二上（2026.9 - 2027.1）详细计划	16

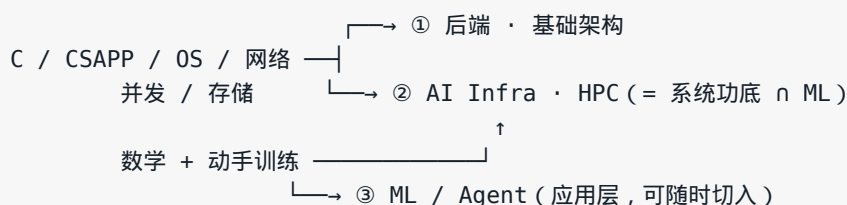
大二下 (2027.2 - 2027.7) 详细计划	16
大三上 (2027.9 - 2028.1): 最关键的一学期	17
大三下 → 秋招 (2028.2 - 2029.1)	17
每周执行模板	17
调整规则 (现实会打乱计划)	17
资源清单与避坑	18
1. 课程 (免费、公开、质量高)	18
2. 书	18
3. 必读源码 (按难度递增, 别跳级)	18
4. 工具箱 (越早熟练越好)	18
5. 避坑清单 (12 条)	19
6. 每月例行 (写进日历)	19
进度追踪	20
0. 每周记录 (每周一行)	20
1. 大二上 · 底座	20
2. 大二上 · 三方向轮转探针	20
3. 大二下 目标 (2027.2 - 2027.7)	20
4. 大三上 目标 (2027.9 - 2028.1)	21
5. 大三下 → 秋招 (2028.2 - 2029.1)	21
6. 决策备忘 (写完粘贴到这里)	21
7. 面试复盘记录	21
8. 「我不喜欢 / 我不擅长」清单 (同样值钱)	21

总览：怎么用这份路线图

起点：2026-09（大二上，约 2029 届毕业） | 时间：每周几乎全部可自由实践 目标：三个方向——① 后端/基础架构 ② AI Infra/高性能计算 ③ 机器学习/Agent，并补齐 CSAPP 等系统基础。

0. 一句话策略

大二上半年的任务不是「选方向」，而是「用作品收集证据」。三个方向共用一条系统底座（C → CSAPP → OS/网络/并发），所以先修底座不会浪费；方向用一个 6 周的「轮转公测」来决定，用评分表决定，不用感觉决定。



三条铁律

1. 任何时刻只有 **1 条主线 + 1 条副线**，外加每天 30 分钟底座。
2. 里程碑必须是**可运行的产出**（repo + benchmark + writeup），不是「看完某课程」。
3. **实习是唯一的硬 deadline**，所有规划倒推自它（见下表）。

1. 硬性时间锚点（倒推规划）

时间	学期	唯一关键目标	验收
2026.9 - 2027.1	大二上	底座 + 三个方向轮转探针	C+CSAPP 4 个 lab、epoll server、3 个探针 repo
2027.2 - 2027.7	大二下	定主攻方向 + 第一个像样的项目 + 投第一份日常实习	决策备忘、一个可讲的项目、面试能过简历关
2027.9 - 2028.1	大三上	主攻方向深挖 + 大厂日常实习	第二/第三个项目、一段实习经历
2027.12 - 2028.4	—	2028 暑期实习投递 （最重要的一次，有转正通道）	拿到 offer
2028.7 - 2028.9	大三暑假	暑期实习，争取转正（return offer）	转正 or 一份大厂背书
2028.8 - 2028.11	大四上	2029 届秋招	上岸

时间充裕的人最容易犯的错：把「准备」无限期延长到秋招前。**从大二下开始就投简历**，用面试反馈校准学习方向，比闭门造车快 10 倍。

2. 文档地图

文件	内容	什么时候看
00 方向选择与轮转公测	先读这个 ：三个探针项目 + 评分表 + 决策规则	现在
01 共同底座-CSAPP 与系统	C、CSAPP 读法与 lab、OSTEP、CS144、15-445、6.824	现在，持续 12 个月
02 主线 A-后端与基础架构	后端方向的技能阶梯与项目	探针期 / 定方向后
03 主线 B-AI-Infra 与 HPC	CUDA/Triton、分布式训练、推理系统	探针期 / 定方向后
04 主线 C-机器学习与 Agent	从零训练、对齐、Agent 系统与评测	探针期 / 定方向后
05 周计划与里程碑	分学期目标、每周节奏模板、验收 gate	每学期初
06 资源与避坑	书/课/源码/工具清单 + 12 条避坑	随时查
progress	可勾选进度表	每周五更新

3. 每周节奏模板（时间充裕版，约 25-30h）

时段	内容	小时
每天固定 0.5h	底座：CSAPP 精读 / 算法题 / 英语技术文档	3.5
主线（大块时间 ×3-4 次）	写代码、啃 lab、调性能	14
副线（大块时间 ×1-2 次）	另一方向的探索性学习	5
周五	写 writeup（500-1500 字）+ 更新 progress.md + 推 commit	2
周日	复盘：本周卡点、下周计划（写进文件，不要放脑子里）	1

每周必须留下三样东西：≥5 个 commit、1 篇 writeup、1 个可运行的新东西。这是「时间充裕」唯一不会翻车的使用方式。

4. 关于「三个方向不知道选哪个」

这是**正常的、甚至是正确**的状态——大二不该凭想象押注。请直接去读 00 方向选择与轮转公测，用 6 周、三个 30-50 小时的小项目，把「喜欢」变成数据。三个方向的技能在系统层高度重叠，所以这段时间即使最后转向，投入也不会作废。

额外的现实信息（可能影响你的选择）：

- **① 后端/基础架构**：岗位量最大、最稳，天花板高（存储/数据库/分布式中间件），但纯 CRUD 岗正在被压缩，要做就做「造轮子」的那一侧。
- **② AI Infra/HPC**：当前最稀缺、溢价最高，门槛也最高（要真会 C/CUDA 和体系结构），本科生能出手的人极少。
- **③ ML/Agent**：岗位多但竞争最卷，且**纯应用层容易被上层框架变化清洗**；真正抗跌的是「能把模型训出来/训得快/评测得住」的人。
- 三者不是互斥：**②=①+ML**，现实岗位多是组合（如「推理引擎工程师」「训练框架工程师」「Agent 平台工程师」）。最稳的路径是 **A（打底）→ B（差异化）**，C 作为随时可切入的副线。

方向选择：三方向轮转公测

目标：用 **6 周 / 3 个探针项目** 把「我好像喜欢 X」变成「数据显示我适合 X」。适用：2026.9-2027.1（大二上）。做完再选，不许提前下结论。

0. 为什么不要现在选

- 你对三个方向的想象，全部来自二手信息（帖子、视频、别人说）。没有亲手做过 **30 小时**，喜欢就是幻觉。
- 大二是唯一一段「选错方向成本 ≈ 0 」的时间窗口：没有实习、没有 KPI、没有 deadline。
- 三个方向 **共享同一条底座**（C / CSAPP / OS / 网络 / 并发 / 存储），无论最后选哪个，这 6 周都不亏。
- 更重要的是：大二的真正目标不是选对方向，而是养成「能独立交付项目」的能力。这个能力跨方向通用。

1. 轮转公测的三个探针

每个探针的设计原则：**30-50 小时**、**2 周**、必须有一个可运行的东西、必须碰到该方向最「苦」的部分（甜的部分谁都觉得好玩）。顺序：先 A（打地基，同时服务另外两个），再 B，再 C。

探针 A：后端 / 系统（第 1-2 周，约 40h）

任务	说明	碰到什么
A1	用 C 写一个 epoll 的 HTTP/1.1 静态文件服务器，支持 keep-alive	非阻塞 IO、超时、缓冲区管理、EBADF/EPIPE 的真实折磨
A2	用 wrk 压测，画出 QPS/延迟曲线；用 perf 找瓶颈，试着优化一轮	性能工程、火焰图
A3（加分）	用 Go 写一个小服务，注册到 etcd，做一次优雅重启	分布式协调、工程效率

验收：GitHub repo + README（含压测数据与优化前后对比）+ 200 字「哪里最爽 / 最烦」。

苦点检验：你能否忍受为 3% 的 QPS 提升花 5 小时看 perf 输出？

探针 B：AI Infra / HPC（第 3-4 周，约 40h）

任务	说明	碰到什么
B1	用 CUDA 写 reduce、sgemm 两个 kernel，与 cuBLAS 对比 GFLOPS	SIMT、shared memory、occupancy
B2	用 ncu 剖析，报告 Roofline 位置 、是否是 memory-bound	性能建模、指标阅读
B3	用 Triton 写一个 fused LayerNorm，和 PyTorch 对比	kernel DSL、融合
B4（加分）	跑一次 7B 模型的 vLLM 推理，用 nsys 看一次 batch 的 timeline	真实推理系统

验收：repo + 一张 Roofline 图 + 一张 ncu 截图 + 「为什么我的 kernel 只有 cuBLAS 的 X%」分析。

苦点检验：你是否能接受没有教程、只有文档和 **profiler**，靠读指标盲调几条通宵？

探针 C：ML / Agent（第 5-6 周，约 40h）

任务	说明	碰到什么
C1	跟着 Karpathy 的 nanoGPT 手写一遍，从零实现 attention + 反向传播	数学落地、张量调试
C2	训一个字符级小模型（能跑通即可），画 loss 曲线	训练循环、诊断
C3	写一个 mini agent：3 个工具（搜索/计算/文件）+ ReAct 循环 + 20 条自建 eval	上下文管理、失败模式
C4（加分）	用 LoRA 微调一个 0.5B 模型做分类任务，对比 zero-shot	对齐、数据工程

验收：repo + loss 曲线 + eval 报告（含失败案例分类）。

苦点检验：你是否受得了结果不确定——调半天不知道是数据、超参还是 eval 的问题？

2. 评分表（做完 3 个探针后填，凭当时的真实感受）

每项 1-5 分。第 1-4 项是主观体验，第 5-7 项是客观现实。

维度	A 后端/系统	B AI Infra/HPC	C ML/Agent
1. 心流：做的时候是否忘记时间			
2. 耐受：遇到硬 bug / 看不懂的指标时，是兴奋还是想逃			
3. 自驱：没人教的情况下，我能自己推进多远			
4. 好奇：路上遇到的「为什么」是否真的想去查			
5. 岗位密度（本科可投的实习数量）			
6. 稀缺性（同龄人里能做出来的人有多少，越少越高）			
7. 可积累性（三年后这项技能还值钱吗）			
合计			

决策规则（写死，防止反复横跳）

1. 主观 4 项之和最高的方向 = 候选主攻；相差 ≤ 2 分视为并列。
2. 并列时，用 主观分 $\times$ 稀缺性 决出主攻（岗位密度只用来判断「能不能找到实习」，不用来选兴趣）。
3. 任何方向如果第 2 项（耐受） ≤ 2 分，直接淘汰出主攻——能力和兴趣都可以培养，耐受不能。
4. 决策输出物：**500 字决策备忘**，包含「我选 X，理由是数据，我的退出条件是……」，存到 progress.md 里。
5. 主攻一旦确定，至少投入 9 个月再重新评估；期间允许副线摇摆。

3. 三种合理结局（都不是失败）

结局	适合谁	后续路径
A 为主，B 为辅	喜欢系统的确定性，但想要差异化	最像「AI Infra / 推理引擎 / 训练框架」岗，也是当前性价比最高的组合
B 为主，C 为辅	既爱硬件性能又爱模型	走「模型工程 / 算法工程 / 高性能推理」
纯 A	只想把系统做深	存储、数据库、分布式中间件、云原生 —— 岗位最稳，天花板极高
纯 C	数学好、喜欢不确定性	但必须补上「工程能力」（否则只是调包），建议至少做完探针 A

千万不要：三个都只做了一半，然后在简历上写「熟悉 XXX」。

4. 这 6 周的额外产出（跨方向通用，写进简历）

- 一个能讲 20 分钟的技术故事（选最有代表性的探针，讲清背景 $\rightarrow$ 方案 $\rightarrow$ 数据 $\rightarrow$ 反思）。
- 习惯：每个项目都带 **benchmark 数据** 和 **README**。这个习惯比方向选择重要 10 倍。
- 一份「我不喜欢什么」的清单（同样值钱，能帮你砍掉很多弯路）。

5. 常见反模式

- **✗** 花两周读《深度学习》花书、看 CS231n，就把探针 C 标记为「喜欢」——读书不是探针，动手才是。
- **✗** 探针 A 只用 Python 写，避开 C —— 那测的是「你会不会 Python」，不是「你适不适合系统方向」。
- **✗** 探针 B 只跑别人的 repo，不写 kernel —— 那测的是「你会不会装环境」。
- **✗** 探针没做完就宣布结论。
- **✗** 一直拖着不选，用「我还在探索」当借口。**6 周后必须写决策备忘，哪怕写「我暂时选 A，退出条件如下」。**

共同底座：CSAPP 与系统基础

定位：这是三个方向都要吃的底座，总投入约 12 个月（每天 0.5-1h 的固定时段 + 项目里的「回头补」）。
核心原则：底座不是先修课，是脚手架。做项目 → 卡住 → 按问题回查对应章节，比线性通读快 5 倍、扎实 3 倍。

1. 学习顺序（有依赖关系的正确拓扑序）

- ① C 语言与内存模型 (2-3 周，必须真会，不是"C 语言语法")
↓
- ② CSAPP 第 3/5/6 章 + lab (4-6 周，CPU/优化/存储层次)
↓
- ③ 组装 + 进程 + 虚拟内存 (CSAPP 第 7/8/9 章 + mallocclab/shlab)
↓
- ④ 并发与 I/O (CSAPP 第 10/12 章，配 epoll 服务器，边做边学)
↓
- ⑤ 网络：CS144 (4-6 周，自己写 TCP)
↓
- ⑥ OS 全景：OSTEP 或 6.S081 (8-10 周，选一个)
↓
- ⑦ 存储/数据库：15-445 (8-10 周，方向 A 必做，方向 B/C 可只做前 4 个 lab)
↓
- ⑧ 分布式：6.824 / 6.5840 (10-12 周，全 lab；方向 A 必做，B 强烈建议)

并行原则：⑥⑦⑧ 不要串行等待，可以在「当前主线方向」的项目里用上它们（比如写 KV 时同时吃 15-445 的 B+ 树 lab）。

2. C 语言：为什么绕不开，怎么学

为什么：读 Linux 源码、内核、CUDA、RocksDB、Redis、PyTorch 的 C++ 层——全部要求你能看懂指针、内存布局、ABI、未定义行为。只会 Python/Java 的人在做 AI Infra 时会在第一层就卡死。

阶段	内容	验收
1	指针与数组的真实关系、void*、函数指针、结构体内存对齐	能手写通用的 qsort 风格回调
2	手动内存管理：malloc/free 语义、悬垂指针、double free、内存泄漏检测 (ASan/valgrind)	用 ASan 找出一段故意埋雷代码的所有问题
3	编译链接：.o/.a/.so、符号、nm/objdump/readelf、静态 vs 动态链接	能解释一次 undefined reference 到底发生在哪个阶段
4	系统调用与错误处理：errno、EINTR 重试、fork/exec/wait	手写一个 mini shell（管道 + 重定向）
5	并发：pthread、互斥锁、条件变量、volatile/原子性、内存序入门	手写生产者-消费者队列，无数据竞争 (TSan 验证)

书籍：K&R（薄，当参考）| 《C 陷阱与缺陷》| 《深入理解 C 指针》| **必备实践：**把 CSAPP 的 lab 用 C 全部做掉，这本身就是最好的 C 训练。

3. CSAPP 正确读法（重点章节 + lab 优先级）

不要说「我要把 CSAPP 读完」。说「我要做掉这 6 个 lab，卡住就回去查章节」。

Lab	对应章节	优先级	耗时	学到什么
Data Lab	第 2 章	★★	10-15h	位运算、补码、浮点 IEEE754
Bomb Lab	第 3 章	★★★	10-15h	读汇编、gdb 单步
Attack Lab	第 3 章	★★	8-12h	栈帧、缓冲区溢出、ROP（理解安全的起点）
Cache Lab	第 6 章	★★★	15-20h	存储层次、局部性、矩阵分块—— 直接迁移到 CUDA/性能优化

Lab	对应章节	优先级	耗时	学到什么
Malloc Lab	第 9 章	★★★	20-25h	显式/隐式空闲链表、碎片、sbrk—— 存储引擎的种子
Shell Lab	第 8 章	★★★	15-20h	进程、信号、作业控制、竞态—— 并发直觉的种子
Proxy Lab	第 10/12 章	★★	15-20h	网络编程 + 线程池 + 缓存，最接近「可写进简历的成品」

章节精读优先级：3（汇编）≈ 6（存储层次）≈ 9（虚拟内存）> 5（性能优化）> 8（异常控制流）> 12（并发）> 10（I/O）> 7（链接）> 2（位）。第 1、11 章扫一眼即可；第 4（处理器体系结构）在第一遍可跳过，等要做 AI Infra 时再回来读。

常见错误读法

- ✗ 从第 1 章线性读到第 12 章、每个 lab 都不做 → 3 个月后全部忘记。
- ✗ 直接抄 lab 答案 → 唯一的价值（卡住 3 小时后的顿悟）被你删掉了。
- ✗ 把 CSAPP 当成「操作系统课」→ 它不教调度/文件系统实现，那些在 OSTEP/6.S081 里。
- ✗ 只读中文版不看 lab handout → lab handout 的英文是你在真实工作里的语言。

4. 操作系统：OSTEP vs 6.S081（二选一）

	OSTEP	MIT 6.S081
形式	免费书 + 少量作业	视频 + 5 个 xv6 lab
优点	讲解极清晰、覆盖全、可快速过	lab 硬核，写完真的懂内核
缺点	动手少	门槛高、耗时 100h+
建议	必读（选读调度、虚拟内存、并发、文件系统四部分）	想在系统方向走深就做，或大三上做

产物：不要只读书。最低配：用 C 写一个 **用户态线程库**（makecontext/swapcontext 或 ucontext）+ 一个 **slab 分配器**。

5. 网络：CS144（强烈建议全做）

自己实现一个 TCP 接收/发送方：序列号、滑动窗口、重传、拥塞控制雏形。做完之后，「TCP 为什么这样设计」「为什么要有 Nagle」「为什么延迟会抖动」这些问题会从背诵变成直觉。

配套：《UNIX 网络编程 卷 1》选读（IO 模型、select/poll/epoll）| 《TCP/IP 详解 卷 1》**产物**：探针 A 的 epoll 服务器 + CS144 的 TCP 实现 = 简历上「网络编程」这条不再是空话。

6. 数据库 / 存储：CMU 15-445

方向 A **必做**；方向 B（训练数据管线/Checkpoint）做 BufferPool + B+Tree 即可；方向 C 可跳过。

lab 顺序：Buffer Pool Manager → B+Tree → Query Execution → Concurrency Control（MVCC/锁）。

产出：一个能跑 SQL 的 mini 数据库，或一个自研 LSM 引擎（写路径：MemTable → WAL → SSTable → Compaction）。

7. 分布式：MIT 6.824 / 6.5840

Lab	内容	价值
Lab 1	MapReduce	入门，快速有成就感
Lab 2	Raft （选举/日志/持久化/快照）	全课程核心，面试能讲 20 分钟
Lab 3	KV Server（线性一致 + 日志压缩）	真正的分布式系统
Lab 4	Sharded KV（配置变更 + 分片迁移）	工程复杂度最高，简历最亮

配套：DDIA（读 2 遍：做 lab 前读一遍建立框架，做完再读一遍对应章节）| [Raft 论文](#)（原文比任何博客都清楚）

8. 数学与体系结构（方向 B/C 专用，方向 A 可后置）

科目	要多少	资源
线性代数	方向 C 必须扎实（矩阵乘法、特征值、SVD 直觉）	MIT 18.06 / 3Blue1Brown
概率统计	方向 C 必须（KL 散度、期望、采样、置信区间）	《概率论与数理统计》+ 实践
微积分/最优化	方向 C（链式法则、梯度、凸性直觉）	反向传播手推一遍胜过看十遍书

科目	要多少	资源
计算机体系结构	方向 B 必须(cache、流水线、SIMT、内存带宽)	CSAPP 第 4/5/6 章 + CMU 15-418 或 UIUC ECE408

注意：方向 A 对数学要求很低，不要在数学上无限投入来逃避写代码。

9. 底座的时间预算（别让它吞掉主线）

项目	总时长	安排方式
C 语言（含 mini shell）	40h	每天 1h，前 2 个月
CSAPP 6 个 lab	80-110h	每天 1h + 周末大块，前 5 个月
OSTEP 选读	30h	分散，遇到问题就读
CS144	50h	大二上后段到寒假
15-445	80h	方向 A：大二下；B：只做 2 个 lab
6.824	100h	大二下-大三上，最值得的一段投入
合计	约 380-400h	按每周 4h = 2 年；按每天 1h = 1 年出头

记住：底座的**唯一目标**是让你在主线项目里不卡在「不懂原理」上。如果底座占用了你 60% 以上的时间，说明你在用学习逃避交付。

主线 A：后端开发 · 基础架构

一句话：这个方向的护城河不在「会写业务代码」，而在**能造轮子、能扛量、能定位故障**。反过来说：只写 CRUD 的「后端」，正是被 AI 压缩得最狠的岗位。

1. 能力分层（自下而上，缺一层就会在面试里露馅）

- L4 架构与权衡：CAP、一致性模型选型、容量规划、成本、演进路径
- L3 分布式系统：共识、复制、分片、事务、消息语义、容错与混沌
- L2 存储与并发：LSM/B+树、WAL、MVCC、锁、无锁、内存管理
- L1 单机性能：epoll/io_uring、内存池、零拷贝、缓存友好、profiling
- L0 语言与系统：C/C++/Go、Linux、网络、进程线程、syscall

招聘市场上的真相：L0/L1 决定你能不能拿到面试（很多人在「讲不出一性能优化」就挂了），L2/L3 决定你是「普通后端」还是「基础架构/中间件」（溢价主要在这里）。

2. 阶梯与产出

阶梯 1：单机性能（第 1-3 月，也服务其他两个方向）

任务	产出
C 写 epoll HTTP 服务器：keep-alive、超时、静态文件、日志	repo + wrk 压测曲线
用 perf/火焰图找出瓶颈，做一轮优化（内存池 / 减少 syscall / sendfile）	优化前后 QPS 对比报告
写一个线程池 + 无锁队列	用 TSan 验证无竞争

关键认知：strace -c / perf top / 火焰图 是后端工程师的听诊器，越早学会越好。

阶梯 2：造轮子（第 3-6 月）——四选一，贪多必失

选题	难度	简历含金量	关键知识点
mini Raft KV	★★★	高（6.824 Lab2+3）	共识、日志复制、快照、线性一致读
mini MQ （对标 Kafka 语义）	★★★	高	顺序写、零拷贝、分区、消费者组、至少一次/幂等
mini 存储引擎 / LSM	★★★★	极高	WAL、MemTable、Compaction、Bloom Filter、崩溃恢复
mini 调度器 / RPC 框架	★★★	中高	服务发现、负载均衡、超时重试熔断、代码生成

强烈建议的第一个轮子：mini Raft KV。它有现成的 lab 框架可以对照，且面试可讲性最高。

阶梯 3：生产化（第 6-12 月）——这一步区分「学生项目」和「工程能力」

- 容器化 + **K8s** 部署（Deployment/Service/ConfigMap/探针），做一次滚动发布与回滚。
- **可观测性**：Prometheus + Grafana 面板 + OpenTelemetry 链路追踪；给系统打上指标（QPS、P99、队列深度、错误率）。
- **压测与容量**：定位 P99 毛刺来源（GC？锁竞争？磁盘 fsync？网络重传？）。
- **故障演练**：杀 leader、网络分区（tc netem）、磁盘写满、时钟漂移 —— 记录系统行为与恢复时间。
- 读源码：**etcd → Redis → RocksDB → Envoy**（按复杂度递增，别一上来读 Envoy）。

阶梯 4：架构与权衡（贯穿，靠写 writeup 沉淀）

- 用 DDIA 的框架解释你做过的每个决定：「为什么这里选 CP 不选 AP」「为什么用推不用拉」。
- 能画出你系统的**故障域图**和**容量模型**，并说明单点在哪、怎么破。

3. 必须能答出来的面试题（自测清单）

- 一次 HTTP 请求从网卡到你的 handler，中间发生了什么？延迟主要在哪些环节？
- epoll 的 LT/ET 区别、惊群问题、为什么需要 EPOLLONESHOT？
- Raft 如何保证已提交日志不丢？为什么多数派提交就能保证？
- 线性一致读为什么不能直接读 follower？ReadIndex / lease read 各自解决什么？
- LSM 的写放大/读放大/空间放大如何权衡？为什么需要 Compaction？
- 你的系统 P99 突然从 20ms 涨到 2s，你的排查顺序是什么？（这题几乎决定你能不能过）

4. 语言选择

语言	用途	建议
C	底座、内核、性能	必须
C++	存储/中间件/推理引擎(字节、腾讯、华为的基础架构多用)	选做, 学 RAII/智能指针/移动语义/并发
Go	云原生、后端服务(工程效率最高, 转正实习友好)	推荐主力 , 2 周即可上手
Rust	新中间件、数据库(TiKV/达梦等)	加分项, 别当第一门语言

结论：C (底层理解) + Go (工程交付) 这个组合对本科实习生性价比最高；想进基础架构组再补 C++。

5. 资源

- 课程：[MIT 6.824/6.5840](#) | [CMU 15-445](#) | [CS144](#)
- 书：[DDIA](#) (圣经) | 《UNIX 网络编程》 | 《Linux 高性能服务器编程》(游双) | 《数据密集型应用系统设计》 | 《SRE Google 运维解密》
- 源码：Redis (万行级入门) → LevelDB/RocksDB → etcd → Envoy
- 工具 perf、火焰图、strace、tcpdump/wireshark、wrk/vegeta、fio、Docker/K8s、Prometheus/Grafana、tc netem

6. 方向 A 的毕业作品 (大二下-大三上完成)

一个自研分布式 KV/MQ + K8s 部署 + Grafana 面板 + 压测报告 + 混沌演练记录 + 一篇 3000 字 writeup。这一份东西，足以让本科简历通过大部分后端/基础架构岗的初筛。

主线 B : AI Infra · 高性能计算

一句话：**AI Infra = 系统功底 n 机器学习**。门槛高、供给少、溢价高，本科生能拿出真作品的人极少——这是三个方向里「差异化收益」最大的一个，也是**最不能靠看视频学会**的一个。

1. 能力地图

L5 编译栈与自定义硬件：TorchInductor / TVM / MLIR / Triton 编译器、NPU 适配
 L4 分布式训练与推理系统：TP/PP/DP/EP、NCCL 通信、PagedAttention、调度与 batching
 L3 单卡 Kernel 工程：FlashAttention、融合算子、量化 kernel、CUDA Graph
 L2 并行计算与体系结构：SIMT、内存层次、occupancy、Roofline、Tensor Core
 L1 C/C++ 与 GPU 编程基础：CUDA C、nvcc、nsys/ncu
 L0 ML 基础 + Python：张量、自动微分、Transformer 结构

关键：L0 不能缺（否则你不知道在优化什么），L1/L2 是护城河（大多数 ML 工程师卡在这里）。

2. 阶梯与产出

阶梯 0：ML 基础（3-4 周，可与主线 C 共用）

- 会写：手推反向传播 + 用 numpy 手写一个两层 MLP 的训练循环。
- 懂结构：Transformer 的每个张量 shape、attention 的复杂度来源、KV Cache 为什么存在。
- 产出去处：见 □主线 C：机器学习·Agent□ 的 C1。

阶梯 1：CUDA 编程（6-8 周，硬核期）

任务	产出
官方 CUDA C Programming Guide 通读 + 示例跑通	笔记
kernel 三件套：vector add → reduce （多级归约）→ sgemm （tiling + shared memory + 寄存器分块）	repo + 与 cuBLAS 的 GFLOPS 对比
用 ncu 分析：occupancy、bank conflict、访存合并、Tensor Core 利用率	每版 kernel 的性能报告
手写 softmax / layernorm / 量化反量化 kernel	融合 kernel 库雏形

核心概念（必须能自己讲清）：warp/warp divergence、shared memory bank conflict、coalesced access、occupancy 与寄存器压力、__syncthreads、异步拷贝（cp.async）、warp shuffle。

阶梯 2：性能建模（2-3 周，决定你是「调参」还是「工程师」）

- Roofline 模型**：算清楚一个算子的 arithmetic intensity，判断它是 compute-bound 还是 memory-bound。
- Amdahl 定律 / 数据搬运成本**：为什么算子融合能赢？为什么 attention 要 flash 化？
- 会用 nsys 看 timeline（kernel 之间的空隙 = 优化空间）、ncu 看单 kernel 指标。
- 产出**：一张自己的 Roofline 图 + 「我的 sgemm 只有 cuBLAS 的 X%，因为……」的分析。

阶梯 3：AI 编译与算子库（4-6 周）

- Triton**：写 fused attention / layernorm / matmul，对比手写 CUDA 的开发效率。
- 读 [FlashAttention](#) 论文 + 源码，理解 tiling 与 online softmax。
- 了解 TorchInductor / TVM / MLIR 的定位（**先了解，别一头扎进编译器，那是另一个大坑**）。

阶梯 4：分布式训练（6-8 周）

内容	关键点
数据并行 / DDP / FSDP	梯度 all-reduce、参数分片、通信-计算 overlap
张量并行 TP / 流水并行 PP / 专家并行 EP	Megatron 的切分方式、PP 的 bubble
集合通信	NCCL 的 ring/tree、all_reduce vs reduce_scatter+all_gather
显存与效率	activation checkpointing、ZeRO 三阶段、混合精度、吞吐（tokens/s/GPU）

产出 : 单机多卡 (哪怕 2 卡) 跑通一次 FSDP , 给出吞吐随卡数/精度/切分方式的对比表 , 指出瓶颈在哪。 **资源** : [Megatron-LM](#)、[DeepSpeed](#)、[PyTorch Distributed 文档](#)、论文《ZeRO》《Megatron-LM》《FlashAttention》。

阶梯 5 : 推理系统 (6-8 周 , 就业最直接的一段)

内容	关键点
vLLM	PagedAttention、continuous batching、前缀缓存 (prefix caching)
SGLang / TensorRT-LLM	RadixAttention、编译优化
量化	GPTQ / AWQ / FP8 / INT4 , 精度-速度权衡
服务指标	TTFT、TPOT、吞吐 (tokens/s)、并发下的延迟曲线、显存占用
自研	自己实现一个最小 continuous batching 调度器 (这是最好的面试话题)

产出 : 一份完整的推理服务调优报告 : QPS-延迟曲线、量化前后对比、batch size 扫描、瓶颈归因。

3. 硬件现实 (决定你能走多远)

条件	能做什么
只有 CPU	能做 : 算子优化 (SIMD/AVX)、量化推理 (llama.cpp)、Roofline 建模、分布式训练的理论及代码阅读
有 1 张消费级 GPU (8-24GB)	能做 : 全部 CUDA/Triton kernel 工作、小模型分布式 (2 卡 FSDP)、7B 量化推理
有云上按小时租的卡 (AutoDL/恒源云等)	上述全部 + 多卡训练实验 ; 按小时计费 , 先本地写好再上机

不要因为「没有卡」就不开始 : 阶梯 1-3 全部只需要一张卡 , 甚至 CPU 上的 SIMD 优化都能学到 80% 的性能工程思维。

4. 自测清单 (面试高频)

- 为什么 shared memory 能加速 ? 什么情况下反而变慢 ? (bank conflict)
- 一个算子 10 TFLOPs , 另一个 200 GB/s 带宽 , 怎么判断哪个更快 ? Roofline 怎么说 ?
- FlashAttention 相比标准 attention , 省的是什么 ? (HBM 读写次数 , 不是 FLOPs)
- 为什么大模型训练必须用 reduce_scatter + all_gather 而不是 all_reduce ?
- PagedAttention 解决了什么问题 ? (KV Cache 的显存碎片与浪费)
- continuous batching 为什么能把吞吐提高数倍 ? 代价是什么 ? (单请求延迟)
- 训练吞吐低 , 你怎么定位 ? (GPU 利用率 → timeline 空隙 → 数据加载 → 通信 → 算子效率)

5. 资源

- 课 : [CMU 15-418](#) / UIUC ECE408 (并行体系结构) | [CUDA C Programming Guide](#)
- 书 : 《Programming Massively Parallel Processors》(PMPP , CUDA 圣经) | 《计算机体系结构 : 量化研究方法》选章
- 工具链 : ncu、nsys、nvprof、cuobjdump、nvidia-smi dmon
- 论文 (按顺序精读) : Attention Is All You Need → FlashAttention v1/v2 → ZeRO → Megatron-LM → PagedAttention(vLLM) → MoE 相关
- 社区 : llm-perf、Triton/CUDA 的 GitHub issue 区 (比教程更真实的问题库)

6. 方向 B 的毕业作品

① 手写 FlashAttention 级 kernel + ncu 报告 ; ② 2 卡 FSDP 吞吐优化报告 ; ③ vLLM 推理服务调优报告 + 自研最小 continuous batching 调度器 ; ④ 一篇拆解论文的 writeup。这套组合在本科实习生里属于前 1%。

主线 C：机器学习 · Agent

一句话：这个方向入门最容易、竞争最激烈、最容易高估自己。抗跌的做法只有一条：能把模型训出来、能评测得住、能端到端上线——而不是会调 API。

1. 能力地图

- L4 系统与落地：推理成本、延迟、评测体系、数据飞轮、监控与回归
- L3 Agent 工程：工具调用、记忆、规划、多智能体、沙箱与安全
- L2 对齐与微调：SFT / LoRA / DPO / 数据构造与清洗
- L1 训练：手写反向传播、训练循环、loss 诊断、分布式训练入门
- L0 原理与数学：线性代数、概率、梯度、Transformer 结构

残酷事实：L0/L1 是筛人的（会手推反向传播的人远少于会 import transformers 的人）；L3/L4 是挣钱的（企业现在真正缺的是「能把 Agent 做稳定并评测出来」的人）。

2. 阶梯与产出

阶梯 0：原理（3-4 周）

- [Karpathy Zero to Hero](#) 全系列：micrograd（手写自动微分）→ makemore → GPT from scratch。
- 配套：手推一遍反向传播（用 numpy 实现两层 MLP，不许用框架自动求导）。
- 产出：一个不依赖 PyTorch autograd 的 MLP 训练脚本 + 一篇推导笔记。
- 数学补课：线代（矩阵乘法/特征值直觉）、概率（KL 散度、采样、期望）、微积分（链式法则）。

阶梯 1：训练（4-6 周）

任务	关键点
nanoGPT 复现并训练一个字符级/小词表模型	训练循环、学习率调度、梯度裁剪、混合精度
换一个真实数据集（如中文语料），做 tokenizer（BPE）	数据处理管线、词表、序列打包
诊断训练：loss 不降 / 震荡 / 过拟合，分别怎么办	这是「会不会做 ML」的分水岭

产出：训练日志、loss 曲线、与 baseline 的对比表、失败实验的记录（失败记录很值钱）。

阶梯 2：微调与对齐（3-4 周）

- **SFT**：小模型（0.5B-3B）指令微调（LLaMA-Factory / trl）。
- **LoRA / QLoRA**：理解低秩假设、rank 与 α 、显存收益。
- **DPO**（及偏好数据构造）；了解 PPO/GRPO 的定位与代价。
- **数据工程**：数据配比、去重、污染检测（eval 集泄漏会让你的成绩一文不值）。
- 产出：一次完整的「数据 → 训练 → 评测 → 迭代」闭环报告，含消融实验。

阶梯 3：Agent 工程（4-6 周，就业最直接）

主题	要点
工具调用	function calling 协议、参数校验、错误恢复、幂等
上下文管理	长上下文策略、压缩/摘要、检索注入、KV 复用
记忆	短期（对话）、长期（向量/结构化）、冲突消解
规划	ReAct / Plan-and-Execute / 反思；何时该放弃自主性改用 workflow
多智能体	分工与通信成本，什么时候它比单 agent 更差
沙箱与安全	代码执行隔离、提示注入、权限边界（企业最关心）
可观测	trace、token 成本、失败归因

产出：一个真实可用的 agent（解决你自己的一个实际问题），带自建 eval 集和失败案例分类。

推荐实现路径：先手写 ReAct 循环（不依赖框架），再对比 LangGraph/其它框架——先手写后才能判断框架是否必要。

阶梯 4：评测与落地（3-4 周，最被低估、最值钱）

- 自建 eval：任务集设计、评分标准、**LLM-as-judge** 的偏差与校准（位置偏差、长度偏差、自我偏好）。
- 指标：pass@k、准确率、鲁棒性、成本（每任务 token/延迟）。
- RAG 全链路：切分 → embedding → 检索（BM25+向量混合）→ rerank → 生成；每一环的评测与消融。

- 上线：监控、回归测试、A/B、prompt/模型版本管理。

3. 自测清单

- 手推 attention 的梯度大致长什么样？为什么需要 $1/\sqrt{d}$ 缩放？
- 训练 loss 正常下降但验证集不降，列出至少 5 个可能原因和排查顺序。
- LoRA 为什么有效？rank 越大越好吗？
- DPO 相比 PPO 省了什么？它依赖什么假设？
- 你的 agent 在任务 A 上成功率 60%，怎么知道提升到 80% 是哪一步的功劳？(消融 + 分层 eval)
- LLM-as-judge 有什么系统性偏差？怎么缓解？
- 一个 Agent 产品的成本从 0.1 元/次涨到 1 元/次，你怎么排查？

4. 资源

- 课：[Karpathy Zero to Hero](#) | [CS231n](#) | [CS224n](#)
- 书：《动手学深度学习》(d2l) | 《深度学习》花书(当参考,别通读) | 《Build a Large Language Model (From Scratch)》(Raschka)
- 工具：PyTorch、HF transformers/trl/peft、LLaMA-Factory、[nanoGPT](#)、vLLM、LangGraph
- 论文：Attention Is All You Need → GPT-3 → InstructGPT → LoRA → DPO → ReAct → Toolformer
→ RAG 相关
- 评测：HELM、lm-evaluation-harness、SWE-bench、GAIA (读题方式比刷分更重要)

5. 方向 C 的毕业作品

① nanoGPT 复现；② 一次完整 SFT→LoRA→DPO 闭环 (含消融)；③ 一个真实可用的 agent + 自建 eval + 失败分析；④ 一份「评测方法论」writeup。

警告：如果毕业作品是「用 LangChain 搭了个问答机器人」，那它在简历上等于零。

6. 方向 C 的最大陷阱

- **X** 只用 API，从不训练 → 无法解释模型行为，面试两个问题就穿。
- **X** 只追框架 → 框架半年一换，原理十年不变。
- **X** 只做 demo 不评测 → 无法回答「你怎么知道它变好了」。
- **X** 逃避数学和系统 → 上限被锁死在「应用层调包」。

周计划与里程碑（大二上 → 秋招）

起点 2026.9（大二上） | 毕业约 2029.6 用法：每学期开学读一次，把当学期的表格抄进 progress.md；每周五花 1 小时更新进度。**Gate** 的含义：不通过 gate 不进入下一学期的大目标。宁可降标准，不可不验收。

学期总览

学期	时间	主题	期末 Gate（硬性验收）
大二上	2026.9-2027.1	底座 + 方向轮转	C 项目 1 个；CSAPP lab ≥ 3 个；三个探针 repo；决策备忘
寒假	2027.1-2	集中突破	CS144 完成；Go 上手
大二下	2027.2-2027.7	定主攻 + 第一个像样项目	6.824 Lab1-2 完成；主攻方向项目 v0.1； 投出第一份日常实习
暑假	2027.7-9	项目深挖 / 实习	项目 v1.0（带 benchmark 与 writeup）
大三上	2027.9-2028.1	大厂日常实习 + 深度	一段实习经历；6.824 Lab3-4 完成； 12 月起投暑期实习
大三下	2028.2-2028.7	暑期实习冲刺	拿到暑期实习 offer；面试复盘 ≥ 10 场
大三暑假	2028.7-9	实习转正	return offer 或强背书
大四上	2028.8-2029.1	秋招	上岸

大二上（2026.9 - 2027.1）详细计划

总时长：约 18 周 $\times$ 25h $\approx$ 450h | 分配：底座 40% / 轮转探针 45% / 输出与复盘 15%

周	主线任务	底座	交付物
1-2	环境搭建（Linux、gcc/gdb、git、Makefile、CMake）；C 语言第 1-5 阶段	CSAPP 第 1-2 章	一份「我的开发环境」笔记
3-4	探针 A：C 写 epoll HTTP 服务器	CSAPP 第 3 章 + Bomb Lab	探针 A repo（含压测）
5-6	探针 A 优化 + Data Lab / Cache Lab	CSAPP 第 6 章	优化对比报告
7-8	探针 B：CUDA reduce + sgemm + ncu 分析	CSAPP 第 5 章	探针 B repo + Roofline 图
9-10	探针 B 续：Triton fused LayerNorm； Malloc Lab	CSAPP 第 9 章	探针 B 报告
11-12	探针 C：nanoGPT 复现 + 手写反向传播	线代补课	探针 C repo + loss 曲线
13-14	探针 C 续：mini agent + 自建 eval； Shell Lab	CSAPP 第 8 章	探针 C eval 报告
15	填评分表 + 写 500 字 决策备忘	—	决策文档 （本目录内）
16-17	选定方向的第一个小项目起步；补 Proxy Lab 或 Attack Lab	CSAPP 第 10/12 章	项目骨架
18	期末复盘：更新本目录、写学期总结	—	学期总结 writeup

期末 Gate 自检

- ☐ 3 个探针 repo 全部公开，每个都有 README + 数据
- ☐ CSAPP lab 完成 ≥ 3 个（建议：Bomb / Cache / Malloc）
- ☐ 决策备忘已写，主攻方向已有数据支撑
- ☐ GitHub 连续提交 ≥ 15 周，writeup ≥ 12 篇

大二下（2027.2 - 2027.7）详细计划

主题：把「会做题」变成「能交付系统」，并且**第一次进入市场**。

月	任务
2	收尾 CS144 的 TCP 实现；Go 语言 2 周速成（写一个小服务）

月	任务
3	6.824 Lab 1 (MapReduce) + Lab 2 (Raft) 起步; 开始投日常实习 (小厂/中厂)
4	Raft 完成 (含快照); 主攻方向项目 v0.1
5	6.824 Lab 3 (KV); 项目加可观测性 / 评测
6	项目 v1.0 + 压测/消融报告 + writeup
7	面试复盘 (≥5 场); 补齐被问倒的知识点; 暑假计划

期末 Gate

- 6.824 Lab 1-2 通过 (Lab 3 加分)
- 主攻方向有一个能讲 20 分钟的项目, 含数据
- 投递 ≥20 家, 拿到 ≥3 场面试 (**哪怕全挂也是成功**——你要的是反馈)
- 能回答 02 / 03 / 04 里对应方向的自测清单 ≥70%

大三上 (2027.9 - 2028.1): 最关键的一学期

两件事并行: ① 日常实习 (争取大厂); ② **12 月起投 2028 暑期实习** (有转正通道, 最重要的一次投递)

月	任务
9	6.824 Lab 3-4; 简历重构 (一页、项目数据化、STAR)
10	投大厂日常实习 (基础架构/中间件/AI Infra/算法工程); 面试
11	进入实习或继续面试; 实习中主动承担「性能/故障」类任务
12	暑期实习投递开启 (阿里/字节/腾讯/美团/华为等), 笔试 + 面试
1	面试复盘; 寒假集中补短板 (按面试反馈, 不按学习计划)

Gate

- 6.824 全 lab 完成 (本方向最强背书之一)
- 有一段实习经历 (日常实习即可)
- 暑期实习投递 ≥15 家, 进入流程 ≥3 家

大三下 → 秋招 (2028.2 - 2029.1)

- **2028.2-4**: 暑期实习面试高峰; 每场面试后 24h 内写复盘 (问了什么、答砸了什么、下周补什么)
- **2028.7-9**: 暑期实习, 目标拿 return offer; 没转正也有大厂背书 + 真实系统经验。
- **2028.8-11**: 秋招。此时你的简历应该是: **1-2 段实习 + 2-3 个带数据的硬核项目 + 6.824/CSAPP 底座**。
- **2029.1**: 补录/春招兜底。

每周执行模板

周一—周五	每天 30–60min 底座 (CSAPP/lab/算法题) 每天 2–3h 主线 (大块时间优先给“要动手写”的任务)
周六	4–6h 副线 (另一个方向的探索)
周日	2h 复盘 + 写 writeup + 更新 progress.md + 推 commit

周检查 (周日问自己)

1. 这周有 commit 吗? ≥5 个?
2. 有一样新东西能跑起来吗?
3. 卡住的点写下来了吗? 下周第一个动作是什么?
4. 有没有「假装学习」(看视频/读文章超过 40% 的时间)? → 立刻调回动手。

调整规则 (现实会打乱计划)

- **某阶段超时 ≤2 周**: 正常, 顺延, 不砍 gate。
- **超时 >4 周**: 说明任务定太大, 把 milestone 砍一半 (砍范围, 不砍验收标准)。
- **拿到实习/面试反馈**: 以反馈为准, **立刻重排优先级**——面试问什么就补什么, 这是最高效的学习信号。
- **对方向产生怀疑**: 允许在学期末重新评估一次, 但必须用新的数据 (新项目/实习体验), 不许凭情绪横跳。

资源清单与避坑

1. 课程（免费、公开、质量高）

课程	方向	说明
CSAPP	全部	系统底座核心，务必做 lab
MIT 6.S081	A/B	xv6 操作系统 lab
OSTEP	A/B	免费操作系统书
CS144	A/B	自己实现 TCP
CMU 15-445	A	数据库系统
MIT 6.824 / 6.5840	A/B	分布式系统，性价比最高的课
CMU 15-418 / UIUC ECE408	B	并行体系结构
Karpathy Zero to Hero	C/B	从零实现 GPT
CS231n / CS224n	C	视觉 / NLP
MIT 18.06	C/B	线性代数

2. 书

书	方向	读法
《深入理解计算机系统》CSAPP	全部	配 lab，别通读
《操作系统导论》OSTEP	A/B	选读四大块
《数据密集型应用系统设计》DDIA	A	读两遍（做 lab 前后各一遍）
《UNIX 网络编程 卷 1》	A/B	选读 IO 模型章节
《Linux 高性能服务器编程》	A	国内面试友好
《Programming Massively Parallel Processors》	B	CUDA 圣经
《计算机体系结构：量化研究方法》	B	选章
《动手学深度学习》d2l	C	边做边读
《Build a Large Language Model (From Scratch)》	C	手写 GPT，极佳
《SRE Google 运维解密》	A	建立工程与可靠性思维
《C 陷阱与缺陷》/《Effective C++》/《Go 语言圣经》	语言	按语言选

3. 必读源码（按难度递增，别跳级）

Redis（万行级，入门首选）

- LevelDB / RocksDB（LSM）
- etcd（Go + Raft，工程典范）
- Envoy（C++，工程复杂度天花板）

nanoGPT（几百行，读得完）

- vLLM（推理系统）
- PyTorch（框架，按需读算子层）
- FlashAttention（kernel 工程）

4. 工具箱（越早熟练越好）

类别	工具
调试	gdb / lldb、rr、ASan/TSan/UBSan、valgrind
性能	perf + 火焰图、strace、ltrace、bpftrace、ncu/nsys
网络	tcpdump、Wireshark、ss、tc netem（故障注入）
压测	wrk、vegeta、fio、locust
构建	Makefile、CMake、Bazel（了解）

类别	工具
环境	Docker、K8s (kind/minikube 起步)、WSL2
可观测	Prometheus、Grafana、OpenTelemetry
ML	PyTorch、HF transformers/trl/peft、vLLM、Triton、W&B/TensorBoard

5. 避坑清单 (12 条)

1. 把「看完课程」当成就。唯一算数的是能跑的东西。看视频时间占比超过 40% 就该警觉。
2. 平摊三个方向。任何时刻 1 主线 + 1 副线，多线同时开 = 全线停在教程第 3 章。
3. 把 **CSAPP** 当先修课线性读完。它是字典 + lab 集合，不是小说。
4. 抄 **lab** 答案。你删掉的正是唯一有价值的部分 (卡住 3 小时后的顿悟)。
5. 只做 **Python**。系统方向的第一道门槛就是 C；不会 C 的 AI Infra 是伪命题。
6. 追框架不追原理。LangChain 半年一换，Raft 和 Roofline 十年不变。
7. 没有 **benchmark** 就说「我优化了」。任何性能/效果结论都必须有前后数据。
8. 不做 **writeup**。写不出来 = 没真懂；而且面试讲的正是 writeup 的内容。
9. 把项目留在本地。不上 GitHub、不写 README 的项目，在简历上等于不存在。
10. 推迟接触市场。大二下就该投简历。面试反馈是最高效的学习信号，闭门造车会走两年弯路。
11. 一直在「准备」岗位需要的技术，却不看 **JD**。每月爬 20 份目标岗位 JD，用词频决定学什么。
12. 用学习逃避交付 / 用交付逃避基础。两者都要有，比例约 3:1 (动手:读书)。

6. 每月例行 (写进日历)

- ☐ 爬取 20 份目标岗位 JD，统计技能词频 → 调整下月优先级
- ☐ 更新 GitHub Profile README (项目列表 + 数据)
- ☐ 读 1 篇论文并写拆解 (方向 B/C 必做)
- ☐ 复盘上月：时间去哪了？(真实计时，不靠感觉)

进度追踪

规则：**每周日更新**。勾选只认「有 commit / 有产出」。不许勾「看完了」。状态标记：[] 未开始 | [~] 进行中 | [x] 完成（附链接/日期）

当前学期：大二上（2026.9 - 2027.1）当前主线：未定（轮转公测中）当前副线：— 上次复盘日期：2026-09-18

0. 每周记录（每周一行）

周	日期	主要产出	commits	writeup	卡点	下周第一动作
1						
2						
3						

1. 大二上 · 底座

- ☐ 开发环境搭建（Linux / gcc / gdb / git / Makefile / CMake）
- ☐ C 语言五阶段（指针→内存→链接→syscall→并发）
- ☐ mini shell（管道 + 重定向）
- ☐ 生产者-消费者队列（TSan 无告警）
- ☐ CSAPP 第 1-2 章 + **Data Lab**
- ☐ CSAPP 第 3 章 + **Bomb Lab**
- ☐ CSAPP 第 3 章 + **Attack Lab**
- ☐ CSAPP 第 5 章（性能优化）
- ☐ CSAPP 第 6 章 + **Cache Lab**
- ☐ CSAPP 第 9 章 + **Malloc Lab**
- ☐ CSAPP 第 8 章 + **Shell Lab**
- ☐ CSAPP 第 10/12 章 + **Proxy Lab**
- ☐ 线代补课（MIT 18.06 选看）

2. 大二上 · 三方向轮转探针

探针 A · 后端/系统

- ☐ A1 C 写 epoll HTTP 服务器（keep-alive / 超时 / 静态文件）
- ☐ A2 wrk 压测 + perf 定位 + 一轮优化（附前后对比）
- ☐ A3（加分）Go 小服务 + etcd 注册 + 优雅重启
- ☐ repo + README + 200 字感受

探针 B · AI Infra/HPC

- ☐ B1 CUDA reduce kernel（多级归约）
- ☐ B2 CUDA sgemm（tiling + shared memory）+ 与 cuBLAS 对比 GFLOPS
- ☐ B3 ncu 剖析 + Roofline 图
- ☐ B4 Triton fused LayerNorm
- ☐ B5（加分）vLLM 跑 7B + nsys timeline
- ☐ repo + 分析报告

探针 C · ML/Agent

- ☐ C1 手写反向传播（numpy，不用 autograd）
- ☐ C2 nanoGPT 复现 + 训练一个字符级模型
- ☐ C3 mini agent（3 个工具 + ReAct 循环 + 20 条 eval）
- ☐ C4（加分）LoRA 微调 0.5B 做分类，对比 zero-shot
- ☐ repo + loss 曲线 + eval 报告

决策

- ☐ 填写评分表（主观 4 项 + 客观 3 项）
- ☐ 写 500 字决策备忘（选择 / 理由 / 退出条件）
- ☐ 主攻方向确定：_____（日期：_____）

3. 大二下 目标（2027.2 - 2027.7）

- ☐ CS144 完成（自己实现 TCP）
- ☐ Go 语言上手（能写小服务）

- ☐ 6.824 Lab 1 MapReduce
- ☐ 6.824 Lab 2 Raft (含快照)
- ☐ 6.824 Lab 3 KV Server
- ☐ 主攻方向项目 v0.1
- ☐ 主攻方向项目 v1.0 (含 benchmark / 消融 / writeup)
- ☐ 投递日常实习 **≥20 家**
- ☐ 面试 ≥3 场 (记录每场问题与复盘)

4. 大三上 目标 (2027.9 - 2028.1)

- ☐ 6.824 Lab 4 Sharded KV
- ☐ 简历重构 (一页 / 项目数据化 / STAR)
- ☐ 日常实习投递 (大厂) + 面试
- ☐ **拿到一段实习经历**
- ☐ 12 月起投 2028 暑期实习 ≥15 家
- ☐ 按面试反馈补短板 (记录清单)

5. 大三下 → 秋招 (2028.2 - 2029.1)

- ☐ 暑期实习面试 (每场 24h 内复盘)
- ☐ 拿到暑期实习 offer
- ☐ 暑期实习 + 争取 return offer
- ☐ 秋招投递与上岸

6. 决策备忘 (写完粘贴到这里)

日期：
 主攻方向：
 主攻理由 (用数据)：
 副线：
 退出条件 (什么情况下我会重新评估)：
 三个月后的里程碑：

7. 面试复盘记录

日期	公司/岗位	被问倒的问题	24h 内的补救动作	完成
----	-------	--------	------------	----

8. 「我不喜欢 / 我不擅长」清单 (同样值钱)

- (例：不喜欢只调参不看指标；不喜欢长时间没有确定性的调试.....)